Let’s say that the word `w` we are checking has `m` characters, and we can accept suggestions differing for at most `k` characters from `w`: in other words, we want words whose *Levenshtein distance* (also known as edit distance) is at most `k`.

To find those words in a trie, we start traversing the tree from the root, and while traversing it we keep an array of `m` elements, where the `i`-th element of this array is the smallest edit distance necessary to match the key corresponding to current node to the first `i` characters in our search string.

For each node `N`, we check the array holding the edit distances.

- If all distances in the array are greater than out maximum tolerance, then we can stop, no need to traverse its subtree any further (as the distances can only grow).
- Otherwise, we keep track of the last edit distance (the one for the whole search string), and if it's the best we have found so far, we pair it with current node's key and store it.

When we finish traversing, we will have saved the closest key to our search string and its distance.

Figure 6.22 shows how the algorithm works on a simplified example; it uses a trie, but the same algorithm, with minor changes, can easily be shown and implemented on radix trees: in fact, for this algorithm we are mostly interested in key nodes, not intermediate ones.